

A Comparative Review of Deep Knowledge Tracing, Sandbox Security, and Personalized Learning Path Generation in Educational Software Systems

Shubhi Sahu¹
IT Department
Acropolis Institute of Technology
& Research, Indore
shubhisahu230688@acropolis.in

Puneet kumar²
IT Department
Acropolis Institute of Technology
& Research, Indore
puneetkumar230801@acropolis.in

Shivam Chouhan³
IT Department
Acropolis Institute of Technology
& Research, Indore
shivamchouhan230751@acropolis.in

Shivani Prajapati⁴
IT Department
Acropolis Institute of Technology
& Research, Indore
shivaniprajapat230830@acropolis.in

Prof. PawanK. Gupta*
IT Department
Acropolis Institute of Technology
& Research, Indore
pawangupta@acropolis.in

Abstract - This paper presents a comparative review of three research works that address distinct but complementary challenges in educational software systems: (1) deep knowledge tracing integrated with cognitive load estimation for personalized learning path generation, (2) deep knowledge tracing applied to open-ended programming exercises using recurrent neural networks, and (3) security analysis of online judge systems using sandbox technology and threat modeling. By examining these works side by side, this review identifies the methodologies, contributions, limitations, and future directions of each, and discusses how they converge toward the broader goal of building intelligent, secure, and adaptive educational technology platforms. The review demonstrates that advances in machine learning, security engineering, and cognitive modeling collectively represent the foundation for next-generation educational software.

Keywords - Deep Knowledge Tracing, Cognitive Load Estimation, Personalized Learning, Online Judge Systems, Sandbox

Security, Neural Networks, Adaptive Learning, Educational Data Mining

I. INTRODUCTION

The intersection of machine learning, cognitive science, and software security has opened new frontiers in educational technology. As institutions increasingly rely on digital platforms for teaching and assessment, the need for intelligent systems that can adapt to individual learners, predict performance accurately, and operate securely has never been greater. This review examines three recent research works that each address a specific dimension of this challenge.

The first paper, by Tong and Ren (2025), proposes a dual-stream neural network architecture that simultaneously performs Deep Knowledge Tracing (DKT) and Cognitive Load Estimation (CLE) to generate personalized learning paths. The second paper, by Wang et al. (2017), extends DKT to the domain of open-ended programming exercises using recurrent neural networks and program embeddings. The third paper, by Kuo et al. (2023), takes a system-security perspective and investigates vulnerabilities in online judge (OJ) systems, proposing a threat model and static code analysis tool based on the STRIDE framework.

While these works span different technical domains, they are united by a shared concern: improving the quality, reliability, and fairness of educational software experiences. Together, they highlight the multi-dimensional nature of educational technology research, which must address not only how students learn, but also

how systems can be made secure, scalable, and responsive to individual cognitive needs.

The remainder of this review is structured as follows. Section II surveys the background literature relevant to each paper. Section III describes the methodologies of the three works. Section IV presents a comparative analysis. Section V discusses results and findings. Section VI identifies common limitations and future directions. Section VII concludes the review.

II. LITERATURE REVIEW

A. Knowledge Tracing and Adaptive Learning

Knowledge tracing, the task of modeling how a student's mastery of skills evolves over time, has a long history rooted in Bayesian Knowledge Tracing (BKT) introduced by Corbett and Anderson [1]. BKT models knowledge as a binary hidden state updated via a Hidden Markov Model. While principled and interpretable, BKT struggles with complex skill interdependencies. The landmark work of Piech et al. [2] introduced Deep Knowledge Tracing (DKT), replacing probabilistic state machines with LSTM-based recurrent neural networks capable of learning rich, continuous knowledge state representations from raw student interaction sequences. Subsequent advances include DKVMN [3], which employs a key-value memory structure, and AKT [4], which uses transformer-based attention to capture long-range dependencies and forgetting.

Cognitive Load Theory, introduced by Sweller [5], provides a complementary psychological framework, distinguishing intrinsic, extraneous, and germane loads. Estimation of cognitive load has traditionally relied on physiological sensors (EEG, heart rate, pupil dilation) or subjective self-report instruments such as the NASA-TLX scale. Machine learning approaches have increasingly been used to estimate load from behavioral proxies such as response time, eye tracking, and keystroke patterns.

B. Deep Learning for Programming Education

The application of DKT to programming exercises presents unique challenges. Unlike multiple-choice assessments, code submissions occupy an unbounded solution space, making feature engineering difficult. Wang et al. [6] addressed this by combining recursive neural networks for program embedding with LSTM-based sequence modeling to predict student success on subsequent exercises in the Hour of Code MOOC. This work built on Piech et al.'s earlier program embedding research [7] and demonstrated that the intermediate submission trajectory of a student carries meaningful information beyond binary correctness labels.

C. Security of Online Judge Systems

Online Judge (OJ) systems are widely used in programming courses to automatically evaluate student-submitted code. Their security has been studied since Forisek [8] catalogued attacks including Denial-of-Service (DoS), Time-Of-Check-to-Time-Of-Use (TOCTTOU),

covert channels, and cross-language attacks. Sims [9] proposed user-level and virtual machine defenses, while Mares [10] presented the Isolate sandbox, which uses Linux namespaces and control groups for secure code execution. The STRIDE threat modeling framework [11] and DREAD risk assessment method provide systematic approaches to identifying and prioritizing vulnerabilities in software systems.

III. METHODOLOGY

A. Tong & Ren (2025): Dual-Stream Neural Architecture for DKT-CLE

Tong and Ren propose a dual-stream neural network architecture consisting of a Knowledge State Representation Stream (KSRS) and a Cognitive Load Estimation Stream (CLES). The KSRS encodes student interaction sequences through a bidirectional LSTM followed by a Graph Attention Network (GAT) that models inter-concept prerequisite relationships. The CLES processes multimodal inputs including behavioral metrics, performance indicators, and optional physiological signals through modality-specific encoders and a Transformer.

The two streams exchange information at each time step through gated cross-stream attention vectors, allowing cognitive load context to modulate knowledge state updates and vice versa. The joint output layer produces simultaneous predictions of knowledge mastery probabilities and cognitive load levels. A dual-objective optimization algorithm formulated as a constrained Markov Decision Process uses Monte Carlo Tree Search to generate personalized learning paths that balance knowledge acquisition with cognitive sustainability.

Experiments used three datasets: a programming platform with 2,183 students, a mathematics application with 862 participants, and a controlled laboratory study (n=124) with EEG-grounded cognitive load labels. The proposed DKT-CLE system was compared against five baselines ranging from rule-based sequencing to RL-based optimization.

B. Wang et al. (2017): DKT on Programming Exercises

Wang et al. apply DKT to a novel setting: open-ended programming exercises from Code.org's Hour of Code MOOC. Each student submission is represented as an Abstract Syntax Tree (AST), which is then vectorized using a recursive neural network trained to predict program outputs. These program embeddings form a sequential trajectory that is fed into an LSTM. The final hidden state is passed through a fully connected softmax layer to predict whether the student will succeed on the next exercise.

To manage the unbounded solution space, the authors control for trajectory length by training separate models for trajectories of lengths 2 to 10. The baseline model uses two hand-crafted features: a Poisson path score estimating the rarity of the student's code trajectory, and a binary indicator of whether the current exercise was solved. Experiments

are conducted on 81,880 students drawn from 263,569 total users of Exercise 18.

C. Kuo et al. (2023): Security Analysis of Online Judge Systems

Kuo et al. adopt a systematic threat-modeling approach to assess OJ security. The Microsoft STRIDE framework is used to classify threats into six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. The DREAD method assigns risk scores to each identified threat. Four threat categories are identified as particularly relevant to OJ systems: resource consumption, disruption of system operation, over-privileged accounts, and sensitive information leakage.

A static code analysis tool is implemented using SonarJava to inspect OJ Java source code for unsafe Isolate sandbox configurations. The tool parses code into ASTs and applies rule processors to detect missing resource constraints (execution time, memory, disk) across five parameter types: STRING_LITERAL, PLUS, IDENTIFIER, NEW_ARRAY, and METHOD_INVOCATION. Five versions of a sandbox application are used as test cases, covering all command-passing methods and sandbox usage scenarios including control group enabled/disabled and multi-process allowed/disallowed configurations.

IV. COMPARATIVE ANALYSIS

Table I below summarizes the three papers across key dimensions: focus area, core technique, principal contribution, and primary limitation.

TABLE I. COMPARATIVE SUMMARY OF REVIEWED PAPERS

Paper	Focus Area	Technique	Key Contribution	Limitation
Tong & Ren (2025)	Adaptive Learning	Dual-stream BiLSTM + Graph Attn.	Integrated DKT-CLE; 87.5% accuracy, 24.6% efficiency gain	No explicit emotional state modeling
Wang et al. (2017)	Programming Education	LSTM + Recursive NN	DKT on open-ended code exercises; 5% improvement over baseline	Single exercise scope; preliminary results
Kuo et al. (2023)	OJS Security	Static Code Analysis + STRIDE	Sandbox threat model; code analysis tool for DoS detection	Java/Isolate only; partial threat coverage

All three works share a common foundation in computer science and software engineering applied to education, yet they differ substantially in their problem framing. Tong and Ren [12] and Wang et al. [6] are squarely within the educational data mining and machine learning tradition, while Kuo et al. [13] approach educational software from a systems security perspective. This divergence reflects the multi-disciplinary nature of educational technology, which must simultaneously address pedagogical effectiveness and system robustness.

From a methodological standpoint, Tong and Ren employ the most architecturally complex solution, combining bidirectional LSTMs, graph attention networks, Transformers, and Monte Carlo Tree Search within a single end-to-end trainable framework. Wang et al. adopt a more focused approach, building on established RNN/LSTM architectures and extending them with recursive tree embeddings for structured data. Kuo et al. depart from machine learning entirely, relying on formal threat modeling and static analysis, which provides deterministic and interpretable results but does not generalize beyond the analyzed system.

In terms of data requirements, Tong and Ren's approach demands multimodal data including physiological signals that may be unavailable in many real-world settings. Wang et al. require only raw code submission logs, which are readily available from existing OJ platforms. Kuo et al. require access to system source code, which is available to developers but not to end users or researchers conducting external evaluations.

V. RESULTS AND DISCUSSION

Tong and Ren report that the DKT-CLE system achieves 87.5% prediction accuracy, an F1 score of 0.86, a path quality rating of 4.4/5, and a 24.6% improvement in learning efficiency over the best baseline. Statistical significance was confirmed by paired t-tests ($p < 0.01$). The cross-stream attention mechanism contributed measurably to these gains: integrated models outperformed independent DKT and CLE models particularly in low-data regimes, suggesting that mutual enhancement between the two streams is a genuine benefit.

A particularly noteworthy qualitative finding was the identification of personalized learning sweet spots: individualized optimal ranges of cognitive challenge that maximized both immediate performance and long-term retention. The system also reduced abandoned learning sessions by 43% compared to non-adaptive baselines, and students achieved mastery 24.6% faster than those following fixed curriculum sequences.

Wang et al. report that their LSTM-based trajectory model consistently outperforms the state-of-the-art baseline by approximately 5% across all nine trajectory-length subsets. Although a 5% gain may appear modest, the authors emphasize that this improvement is achieved without any hand-crafted features, relying entirely on learned program embeddings. The program output prediction accuracy of

96% on a held-out set (versus 54% for a majority-class baseline) confirms that the AST-based embeddings capture meaningful semantic structure of student code.

Kuo et al. demonstrate that their code analysis tool successfully identifies all five categories of sandbox misconfigurations across the tested application versions, covering all five parameter types. In practical terms, the tool flagged missing execution time, memory, and disk limits that would allow a malicious student submission to exhaust system resources via a DoS attack. When control groups were enabled but memory limits were not applied at the group level, the tool correctly identified that per-process limits were insufficient to prevent multi-process resource exhaustion, as confirmed by experimental runs that consumed over 400 MB of RAM.

Taken together, these results point to a consistent theme: the combination of richer data representations and more sophisticated modeling consistently yields better outcomes, whether the goal is predicting student performance, generating adaptive learning paths, or detecting security vulnerabilities. In each case, the naive or manual baseline was substantially outperformed by the proposed system.

VI. LIMITATIONS AND FUTURE DIRECTIONS

Several limitations are common across the reviewed works. First, scalability and generalizability remain open questions. Tong and Ren's dual-objective optimization is computationally intensive, raising concerns for deployment at scale. Wang et al.'s approach is evaluated on a single exercise from a single MOOC. Kuo et al.'s tool supports only Java-based systems using the Isolate sandbox, leaving other languages and sandbox environments uncovered.

Second, the three works address orthogonal dimensions of educational software in isolation. Tong and Ren do not consider the security of the adaptive learning platform they propose. Wang et al. do not discuss the privacy risks of collecting detailed code submission telemetry. Kuo et al. do not consider how security measures interact with the learning experience of students who may be penalized by overly restrictive sandboxes.

Third, each work acknowledges the challenge of data quality. Tong and Ren note that physiological sensor data is often unavailable, and behavioral proxies are noisy. Wang et al. work with data from only two exercises due to availability constraints. Kuo et al. note that their tool can only analyze code available to developers, and cannot detect runtime attacks or zero-day vulnerabilities.

Future directions identified by the authors include: integrating emotional and motivational state detection into adaptive learning models; extending program embedding approaches to multi-exercise trajectories; expanding the code analysis tool to support Python, C++, and alternative sandboxes; incorporating explainable AI techniques to improve transparency; and developing real-time

dashboards that unify learning analytics with security monitoring. An integrated framework that addresses all three dimensions simultaneously, i.e., one that is cognitively adaptive, educationally effective, and operationally secure, represents a compelling and as-yet-unrealized research goal.

VII. CONCLUSION

This review has examined three research works that collectively address the multi-dimensional challenges of modern educational software: personalized learning through deep knowledge tracing and cognitive load estimation, performance prediction in open-ended programming environments, and security hardening of online judge systems. Each work makes a distinct and valuable contribution to the field, and each identifies important limitations that point toward productive future research.

The first work demonstrates that simultaneously modeling cognitive load and knowledge state yields more effective and engaging learning paths than either approach alone. The second work demonstrates that meaningful progress representations can be learned from raw code submissions without domain-specific feature engineering. The third work demonstrates that systematic threat modeling and static analysis can surface exploitable sandbox misconfigurations before they are attacked in production.

Viewed together, these works underscore that building effective educational software is not merely a machine learning problem, nor a security problem, nor a pedagogical problem in isolation. It is an inherently interdisciplinary challenge that demands concurrent attention to how students learn, how systems can be made adaptive, and how platforms can be made secure. Progress on all three fronts is necessary before intelligent educational systems can be responsibly deployed at scale.

REFERENCES

- [1] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.
- [2] C. Piech et al., "Deep knowledge tracing," *Advances in Neural Information Processing Systems*, pp. 505–513, 2015.
- [3] J. Zhang et al., "Dynamic key-value memory networks for knowledge tracing," *Proceedings of the 26th International Conference on World Wide Web*, pp. 765–774, 2017.
- [4] A. Ghosh, N. Heffernan, and A. S. Lan, "Context-aware attentive knowledge tracing," *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2330–2339, 2020.
- [5] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.
- [6] L. Wang, A. Sy, L. Liu, and C. Piech, "Deep knowledge tracing on programming exercises," *Proceedings of the 4th ACM Conference on Learning at Scale (L@S)*, pp. 201–204, 2017.

- [7] C. Piech et al., "Learning program embeddings to propagate feedback on student code," CoRR abs/1505.05969, 2015.
- [8] M. Forisek, "Security of programming contest systems," Information Technology in Schools, pp. 553–563, 2006.
- [9] R. W. Sims, "Secure execution of student code," Technical Report, University of Maryland, 2012.
- [10] M. Mares and B. Blackham, "A new contest sandbox," Olympiads in Informatics, vol. 6, pp. 100–109, 2012.
- [11] Microsoft, "The STRIDE Threat Model," Microsoft Docs, 2022. [Online]. Available: <https://docs.microsoft.com/en-us/previous-versions/commerce-server/ee823878>
- [12] C. Tong and C. Ren, "Deep knowledge tracing and cognitive load estimation for personalized learning path generation using neural network architecture," Scientific Reports, vol. 15, no. 24925, 2025.
- [13] J.-Y. Kuo, Z.-J. Wen, T.-F. Hsieh, and H.-X. Huang, "A study on the security of online judge system applied sandbox technology," Electronics, vol. 12, no. 3018, 2023.